

```

// Business Discovery Engine – motor de busca de empresas SEM depender diretamente
// do Google Places API (que exige cartão). Usa a Local Business Data API da
// OpenWeb Ninja, que tem um nível gratuito real (500 empresas/mês, sem cartão)
// e devolve dados vindos do Google Maps – nome, telefone, endereço, avaliação
// e fotos – através de uma chave x-api-key obtida de graça em openwebninja.com.
//
// Configure no .env:
//   OPENWEBNINJA_API_KEY=sua_chave_gratuita
//   PEXELS_API_KEY=sua_chave_gratuita    (fallback de imagens genéricas)
//   PORT=8787
//   ALLOWED_ORIGIN=*

import express from 'express';
import cors from 'cors';

const app = express();
app.use(express.json({limit:'1mb'}));
app.use(cors({origin: process.env.ALLOWED_ORIGIN || true}));

const PORT = Number(process.env.PORT || 8787);
const OWN_KEY = process.env.OPENWEBNINJA_API_KEY || '';
const PEXELS_KEY = process.env.PEXELS_API_KEY || '';
const OWN_SEARCH_URL = 'https://api.openwebninja.com/local-business-data/search';
const PEXELS_SEARCH_URL = 'https://api.pexels.com/v1/search';

const clean = v => String(v ?? '').trim();
const uniq = arr => [...new Map(arr.filter(Boolean).map(x => [x.url || JSON.strin

function relevance(item, niche, city){
  const text = `${item.name||''} ${item.address||''} ${item.city||''} ${item.nich
  let score = 40;
  if (city && text.includes(city.toLowerCase())) score += 25;
  if (niche && text.includes(niche.toLowerCase())) score += 25;
  if (item.website) score += 5;
  if (item.photos?.length) score += 5;
  return Math.min(100, score);
}

// Empresa real vinda da Local Business Data API (formato próprio da OpenWeb Ninj
// normalizada pro formato que o frontend do AI Website Hunter espera.
function normalizeBusiness(x, niche, city){
  const photos = uniq([
    ...(Array.isArray(x.photos_sample) ? x.photos_sample : []),
    ].map(p => typeof p === 'string' ? {url:p} : {url: p?.photo_url_large || p?.pho

```

```

return {
  id: x.business_id || x.place_id || `biz-${Math.random().toString(36).slice(2)}`,
  name: x.name || 'Empresa encontrada',
  niche: niche,
  city: x.city || city || '',
  country: 'Brasil',
  address: x.full_address || x.address || '',
  phone: x.phone_number || '',
  website: x.website || '',
  sourceUrl: x.place_link || '',
  source: 'Google Maps (via OpenWeb Ninja)',
  photos: photos.map(p => p.url).filter(Boolean),
  instagram: x.emails_and_contacts?.instagram || '',
  facebook: x.emails_and_contacts?.facebook || '',
  whatsapp: '',
  hasWebsite: !!x.website,
  rating: x.rating ?? null,
  ratingCount: x.review_count ?? 0,
  openingHours: x.working_hours ? Object.entries(x.working_hours).map(([d,h])=>
  openNow: x.opening_status ? /open/i.test(x.opening_status) : null,
  real: true,
  live: true,
  relevance: relevance({name:x.name, address:x.full_address, website:x.website,
  score: relevance({name:x.name, address:x.full_address, website:x.website, pho
});
}

app.get('/api/health', (req,res)=>res.json({
  ok:true,
  discoveryEngineConfigured: !!OWN_KEY,
  imageSearchConfigured: !!PEXELS_KEY,
  provider: OWN_KEY ? 'openwebninja-local-business-data' : 'not-configured',
  pexelsConfigured: !!PEXELS_KEY,
  googlePlacesConfigured: !!OWN_KEY, // compatibilidade com o frontend, que exige
  openAiConfigured: false,
}));

app.post('/api/companies/search-advanced', async (req,res)=>{
  try{
    const b=req.body||{};
    const niche=clean(b.nicho);
    const city=clean(b.cidade);
    if(!OWN_KEY) return res.status(503).json({companies:[],error:'Configure OPENW

    const query = [niche && niche !== 'Todos os tipos de empresa' ? niche : 'empr
    const limit = Math.min(20, Number(b.quantidade || 25)); // free tier: use com

```

```

const url = new URL(OWN_SEARCH_URL);
url.searchParams.set('query', query);
url.searchParams.set('limit', String(limit));

const r = await fetch(url, { headers: { 'x-api-key': OWN_KEY } });
if(!r.ok) throw new Error(`OpenWeb Ninja retornou ${r.status}`);
const data = await r.json();
const raw = data.data || data.results || [];

const companies = uniq(raw.map(x=>normalizeBusiness(x, niche, city)))
  .sort((a,b)=>b.relevance-a.relevance)
  .slice(0, limit);

res.json({ companies, count: companies.length, requested: limit, insufficient
} catch(e){res.status(502).json({companies:[],error:e.message});}
});

// Imagens de banco (Pexels) - só usadas como reforço quando a empresa não tem
// fotos suficientes vindas do Google Maps.
app.get('/api/images/search', async (req,res)=>{
  try{
    const q=clean(req.query.query);
    const count=Math.min(20,Math.max(1,Number(req.query.count||6)));
    if(!PEXELS_KEY) return res.json({images:[]});
    const url = new URL(PEXELS_SEARCH_URL);
    url.searchParams.set('query', q);
    url.searchParams.set('per_page', count);
    url.searchParams.set('orientation', 'landscape');
    const r = await fetch(url, { headers: { Authorization: PEXELS_KEY } });
    if(!r.ok) throw new Error(`Pexels retornou ${r.status}`);
    const data = await r.json();
    const images = (data.photos||[]).map(p=>({
      url: p.src.large, thumb: p.src.medium, photographer: p.photographer, source
    }));
    res.json({images});
  } catch(e){res.status(502).json({images:[],error:e.message});}
});

app.listen(PORT, ()=>console.log(`Business Discovery Engine ativo em http://localh

```